

K-디지털 트레이닝

바이트 코딩와 AI 페어 프로그래밍, GeminiCLI

[KB] IT's Your Life

✓ 바이브 코딩 (Vibe Coding)

- 개발자가 상세한 로직이나 문법을 고민하기보다, 추상적인 의도와 '느낌(Vibe)'을 전달하여 AI가 코드 전체를 생성하게 만드는 방식
- 특징
 - 결과 중심
 - "이런 기능을 가진 대시보드 만들어줘"처럼 자연어로 의도를 던지고,
 - AI가 뱉어낸 결과물이 제대로 돌아가는지에 집중
 - 추상화
 - 내부 구현 방식(어떤 라이브러리를 쓰는지, 변수명이 무엇인지 등)보다는 전체적인 시스템의 동작과 사용자 경험을 우선시
 - 반복적 수정(Iterative Tweak)
 - 코드가 마음에 안 들거나 버그가 나면 직접 수정하기보다 "이 부분 좀 더 세련되게 바꿔줘", "에러 나니까 고쳐줘"라고 다시 AI에게 시킴
- 장점
 - 비전공자나 아이디어를 빠르게 검증하려는 기획자에게 유리하며, 프로토타입 제작 속도가 압도적으로 빠름

✓ AI 페어 프로그래밍 (AI Pair Programming)

- 전통적인 페어 프로그래밍에서 파트너(네비게이터) 역할을 AI가 대신하는 방식
- 개발자가 코드의 설계와 품질을 주도하되, AI를 강력한 보조 도구로 활용
- 특징
 - 과정 중심
 - 개발자가 로직을 설계하고 코드를 작성하는 과정에서 AI에게 "이 함수 최적화해줘", "테스트 코드 짜줘"와 같이 구체적인 도움을 요청
 - 코드 소유권
 - 생성된 코드의 한 줄 한 줄을 개발자가 완벽히 이해하고 검토(Review)하며,
 - 최종적인 아키텍처 결정권을 사람이 가짐
 - 도구로서의 AI
 - 복잡한 알고리즘 구현, 상용구 코드(Boilerplate) 작성, 문서화 등 귀찮고 반복적인 작업을 AI에게 맡김
- 장점
 - 코드의 유지보수성이 높고, 대규모 프로젝트나 기업용 소프트웨어 개발에 적합

✓ 주요 차이점 비교

구분	바이브 코딩 (Vibe Coding)	AI 페어 프로그래밍
핵심가치	속도와 실행 (Just work it!)	품질과 제어 (Clean & Stable)
개발자의 역할	프롬프트를 던지는 디렉터(Director)	코드를 검토하고 승인하는 리뷰어(Reviewer)
코드 이해도	결과물만 확인 (내부 로직은 몰라도 됨)	모든 코드의 동작 원리를 파악함
주요 도구	Cursor(에이전트 모드), Replit Agent	GitHub Copilot, Tabnine
적합한 상황	1인 개발, MVP(최소 기능 제품) 제작	팀 프로젝트, 대규모 서비스, 장기 유지보수

✓ Gemini CLI

- 구글의 가장 강력한 AI 모델인 Gemini를 터미널(Terminal) 환경에서 직접 사용할 수 있도록 해주는 오픈소스 AI 에이전트
 - 단순한 챗봇을 넘어, 개발자의 로컬 파일 시스템에 접근하거나 셸 명령어를 실행하고,
 - 웹 검색을 수행하는 등 실제 코딩 작업에 깊숙이 관여할 수 있는 기능을 갖추
- 주요 특징 및 기능
 - 터미널 통합
 - IDE를 벗어나지 않고도 터미널에서 즉시 코드 분석, 디버깅, 문서 생성이 가능
 - ReAct 루프 (Reasoning and Acting)
 - 단순히 답만 하는 것이 아니라, 필요에 따라 "파일 읽기 -> 분석 -> 셸 실행 -> 결과 확인"과 같은 일련의 과정을 스스로 계획하고 실행
 - 강력한 컨텍스트 활용
 - GEMINI.md 파일을 통해 프로젝트별 가이드라인이나 코딩 컨벤션을 AI에게 미리 학습시켜 일관된 응답 유도 가능
 - MCP(Model Context Protocol) 지원
 - 외부 툴이나 데이터 소스를 연결하여 기능 확장 가능
 - 멀티모달 지원
 - 터미널임에도 불구하고 로컬의 PDF, 이미지 등을 읽어 분석하거나 이를 바탕으로 앱을 생성할 수 있음

✓ Gemini CLI

- 무료 사용량
 - 분당 60회 요청
 - 하루 1,000회 요청

✓ Gemini CLI 설치

○ 요구조건

- Git Bash
- Node.js
- 파이썬 → 데이터 가공 시 필요

○ 전역 설치

```
$ npm install -g @google/gemini-cli
```

cf] 전역 모듈 설치 위치

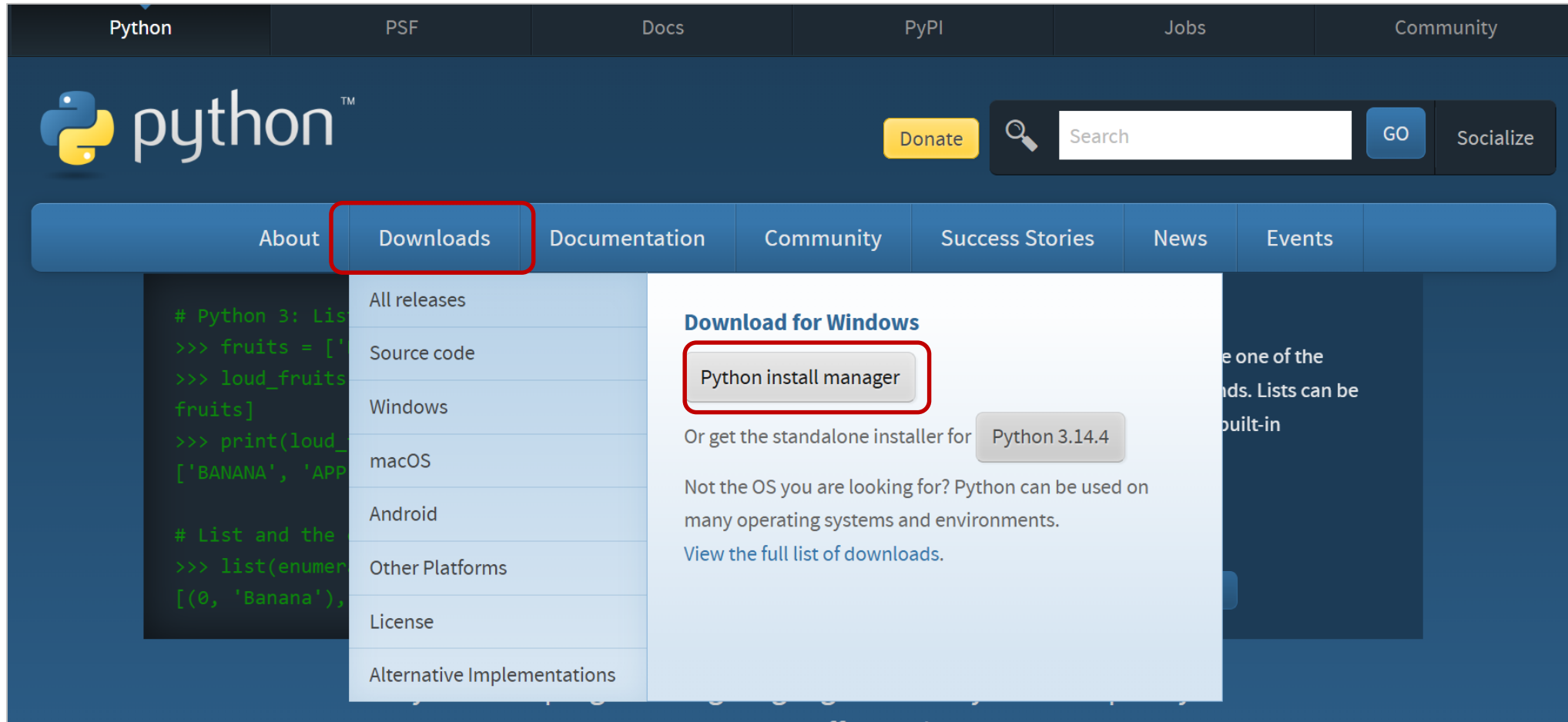
- C:\Users\<User 계정>\AppData\Roaming\npm\node_modules

cf] gemini 전역 설정 파일 위치

- C:\Users\<User 계정>\.gemini

✅ 파이썬 설치

- <https://www.python.org/>



✓ Gemini CLI 실행

```
$ gemini
```

```
| > Do you want to connect IDE to Gemini CLI?
```

```
| If you select Yes, we'll install an extension that allows the CLI to access your open files and display diffs directly in IDE.
```

```
|
```

```
| ● 1. Yes
```

```
| 2. No (esc)
```

```
| 3. No, don't ask again
```

```
| ? Get started
```

```
| How would you like to authenticate for this project?
```

```
| ● 1. Login with Google
```

```
| 2. Use Gemini API Key
```

```
| 3. Vertex AI
```

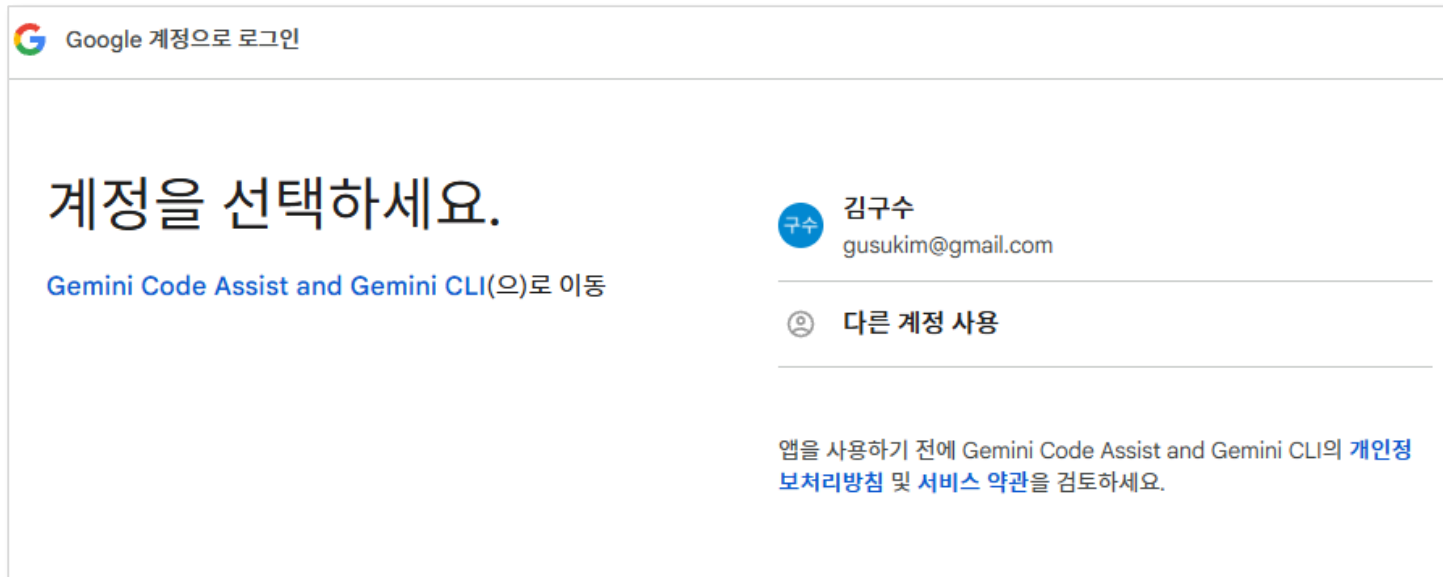
```
| No authentication method selected.
```

```
| (Use Enter to select)
```

```
| Terms of Services and Privacy Notice for Gemini CLI
```

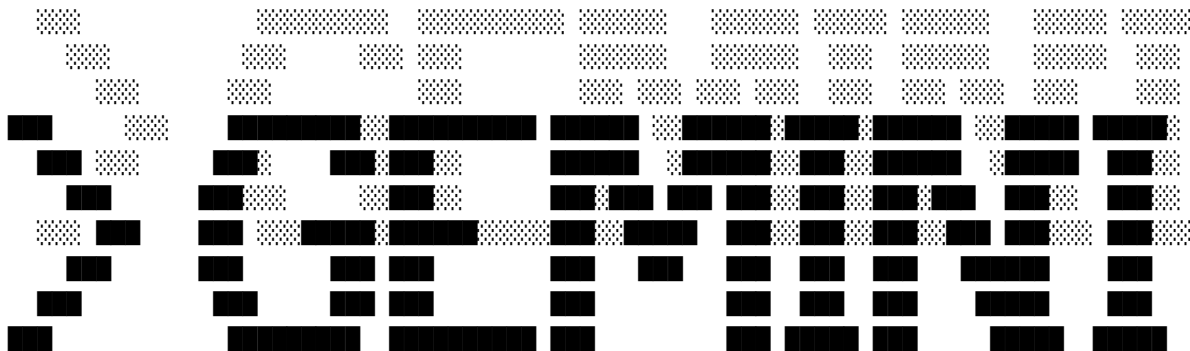
```
|
```

```
| https://github.com/google-gemini/gemini-cli/blob/main/docs/tos-privacy.md
```



! Authentication succeeded

| You have successfully logged in with Google. Gemini CLI needs to be restarted. Press 'r' to restart, or 'escape' to choose a different auth method.





대화 입력창에서 줄바꿈 → Ctrl + J

✔ 종료

```
/quit
```

```
| Agent powering down. Goodbye!
```

```
| Interaction Summary
```

```
| Session ID:                de5046e7-6609-4523-8043-1b915f92c6f7
```

```
| Tool Calls:                1 ( ✓ 0 x 1 )
```

```
| Success Rate:              0.0%
```

```
| Performance
```

```
| Wall Time:                  10m 12s
```

```
| Agent Active:               47.8s
```

```
|   » API Time:                23.0s (48.1%)
```

```
|   » Tool Time:               24.8s (51.9%)
```

```
| Model Usage                  Reqs    Input Tokens    Cache Reads    Output Tokens
```

```
| gemini-2.5-flash-lite        3          2,770           0             144
```

```
| gemini-2.5-flash             3          7,720        11,719         20
```

```
| Savings Highlight: 11,719 (52.8%) of input tokens were served from the cache, reducing costs.
```

✓ Gemini.md

- Gemini CLI가 프로젝트의 성격, 규칙, 그리고 개발자의 의도를 정확히 파악할 수 있도록 돕는 '프로젝트 전용 설명서'.
- 마치 새로운 팀원(AI)에게 "우리 프로젝트는 이런 규칙으로 돌아가니까 이렇게 코딩해줘"라고 전달하는 가이드라인과 같음
- 역할
 - Gemini CLI는 실행될 때 해당 디렉토리의 Gemini.md 파일을 자동으로 읽음
 - 이를 통해 AI는 다음과 같은 컨텍스트를 유지
 - 일관성 유지:
선호하는 코딩 스타일(예: 탭/스페이스, 화살표 함수 사용 등)을 강제
 - 프로젝트 이해:
기술 스택, 폴더 구조, 주요 기능을 미리 알고 대응
 - 실수 방지:
특정 라이브러리 사용 시 주의사항이나 금지된 패턴을 명시

✓ Gemini.md 작성법 (구성 요소)

- Markdown 형식을 사용
- 필수 포함 권장 항목
 - Project Overview
 - 프로젝트의 목적과 주요 기능을 간단히 설명
 - Tech Stack
 - 사용하는 언어, 프레임워크, 라이브러리 버전을 명시
 - Coding Standards
 - 변수 명명 규칙, 코드 구조, 스타일 가이드를 정의
 - Instructions
 - AI가 작업을 수행할 때 지켜야 할 특이 사항 작성

✅ Gemini.md 작성 예시

Project: My Awesome App

📖 Overview

이 프로젝트는 사용자가 식물을 관리할 수 있는 '홈 가드닝 대시보드' 웹 서비스입니다.

⚙️ Tech Stack

- Frontend: Vue.js (Vue Router,), Bootstrap5, Axios, FontAwesome
- State Management: Pinia
- Backend: json-server

📏 Coding Standards

- 모든 컴포넌트는 Javascript를 사용하여 Compsition API로 작성합니다.
- 함수형 컴포넌트와 ****화살표 함수****를 선호합니다.
- 스타일링은 Bootstrap5 클래스만 사용하며, 인라인 스타일은 금지합니다.

AI Instructions

- 코드를 수정할 때 기존의 에러 핸들링 패턴을 유지해주세요.
- 새로운 기능을 만들 때 항상 `components/ui`에 있는 공통 컴포넌트를 먼저 확인하세요.
- 성능 최적화보다는 ****가독성****이 좋은 코드를 우선순위에 둡니다.
- 답변은 가급적 한국어로 설명하되, 기술 용어는 원문을 병행 표기해주세요.

K-디지털 트레이닝

첫 대화 시작하기

[KB] IT's Your Life

✓ 프롬프트

○ 순수 자바스크립트로 할 일 목록 프로그램 만들어줘

+ I will explore the workspace to confirm it is empty and then prepare a plan for your To-Do List application.

✓ ReadFolder . → Directory is empty.

+ I will now enter plan mode to design a modern, functional To-Do List application for you.

Action Required

? Enter Plan Mode Designing a new To-Do List application from scratch.

This will restrict the agent to read-only tools to allow for safe planning.

Do you want to proceed?

1. Allow once
- 2. Allow for this session
3. No, suggest changes (esc)

```
66 };
67
68 // 필터링 이벤트
69 filterBtns.forEach(btn => {
70   btn.addEventListener('click', () => {
71     filterBtns.forEach(b => b.classList.remove('active'));
72     btn.classList.add('active');
73     currentFilter = btn.dataset.filter;
74     render();
75   });
76 });
77
78 // 초기 렌더링
79 render();
```

이제 위 파일들을 해당 폴더에 저장한 후 `index.html`을 브라우저로 열면 기능이 작동합니다. 이 프로그램은 다음과 같은 특징을 가집니다:

1. 데이터 보존: 브라우저를 새로고침해도 `localStorage`를 통해 목록이 유지됩니다.
2. 필터링: 전체, 진행 중, 완료된 항목을 나누어 볼 수 있습니다.
3. 애니메이션: 항목 추가 시 부드러운 페이드인 효과가 적용됩니다.
4. 반응형 디자인: 모바일과 데스크탑에서 모두 보기 좋습니다.

파일을 생성해 드릴까요?

(파일 생성을 원하시면 `write_file` 도구를 사용하여 직접 폴더에 만들어 드릴 수 있습니다.)

✓ 프롬프트

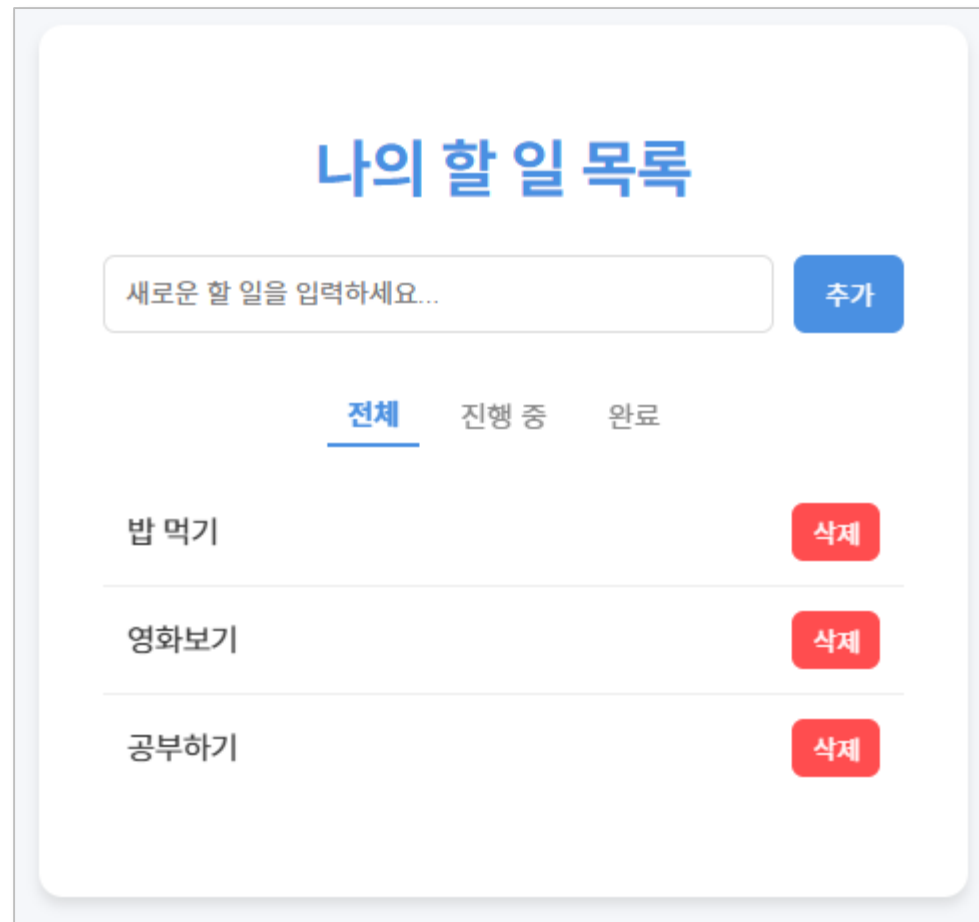
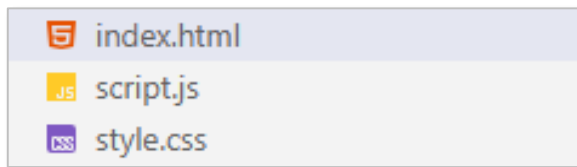
○ 만들어줘.

+ 순수한 자바스크립트로 구성된 할 일 목록 프로그램 제작을 완료했습니다.

생성된 파일:

- `index.html`: 애플리케이션의 기본 구조
- `style.css`: 현대적인 스타일링 및 애니메이션
- `script.js`: 할 일 추가, 삭제, 완료 토글, 필터링 및 로컬 저장소 기능

이제 브라우저에서 `index.html` 파일을 열어 확인해 보세요! 추가로 필요한 기능이 있으시면 말씀해 주세요.



index.html

```
...
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>나의 할 일 목록</h1>
    <form id="todo-form">
      <input type="text" id="todo-input" placeholder="새로운 할 일을 입력하세요..." required>
      <button type="submit">추가</button>
    </form>

    <div class="filters">
      <button class="filter-btn active" data-filter="all">전체</button>
      <button class="filter-btn" data-filter="active">진행 중</button>
      <button class="filter-btn" data-filter="completed">완료</button>
    </div>

    <ul id="todo-list"></ul>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
const todoForm = document.getElementById('todo-form');
const todoInput = document.getElementById('todo-input');
const todoList = document.getElementById('todo-list');
const filterBtns = document.querySelectorAll('.filter-btn');

// 로컬 저장소에서 데이터 불러오기
let todos = JSON.parse(localStorage.getItem('todos')) || [];
let currentFilter = 'all';

// 할 일 추가
todoForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const text = todoInput.value.trim();
  if (text) {
    const newTodo = {
      id: Date.now(),
      text,
      completed: false
    };
    todos.push(newTodo);
    saveAndRender();
    todoInput.value = '';
  }
});
```

script.js

```
// 상태 저장 및 렌더링
function saveAndRender() {
  localStorage.setItem('todos', JSON.stringify(todos));
  render();
}

// 할 일 완료/미완료 토글
window.toggleTodo = (id) => {
  todos = todos.map(todo =>
    todo.id === id ? { ...todo, completed: !todo.completed } : todo
  );
  saveAndRender();
};

// 할 일 삭제
window.deleteTodo = (id) => {
  todos = todos.filter(todo => todo.id !== id);
  saveAndRender();
};
```

script.js

```
// 렌더링 함수
function render() {
  todoList.innerHTML = '';

  const filteredTodos = todos.filter(todo => {
    if (currentFilter === 'active') return !todo.completed;
    if (currentFilter === 'completed') return todo.completed;
    return true;
  });

  filteredTodos.forEach(todo => {
    const li = document.createElement('li');
    li.className = `todo-item ${todo.completed ? 'completed' : ''}`;

    li.innerHTML = `
      <span onclick="toggleTodo(${todo.id})">${todo.text}</span>
      <button class="delete-btn" onclick="deleteTodo(${todo.id})">삭제</button>
    `;
    todoList.appendChild(li);
  });
}
```


script.js

```
// 필터링 이벤트
filterBtns.forEach(btn => {
  btn.addEventListener('click', () => {
    filterBtns.forEach(b => b.classList.remove('active'));
    btn.classList.add('active');
    currentFilter = btn.dataset.filter;
    render();
  });
});

// 초기 렌더링
render();
```

✓ 종료하기

○ 프로그래프트

- /quit

Agent powering down. Goodbye!

Interaction Summary

Session ID: 84ff38db-785b-4545-8d7d-3a297d9a2786
Tool Calls: 10 (✓ 8 x 2)
Success Rate: 80.0%
User Agreement: 88.9% (9 reviewed)
Code Changes: +279 -0

Performance

Wall Time: 15m 7s
Agent Active: 7m 29s
» API Time: 4m 1s (53.6%)
» Tool Time: 3m 28s (46.4%)

Model Usage

Use /model to view model quota information

Model	Reqs	Input Tokens	Cache Reads	Output Tokens
gemini-3-flash-preview	13	161,517	51,661	5,360
↳ main	13	161,517	51,661	5,360

To resume this session: `gemini --resume '84ff38db-785b-4545-8d7d-3a297d9a2786'`

K-디지털 트레이닝

효과적인 프롬프트 작성하기

[KB] IT's Your Life

✓ 프롬프트 구성요소

- 페르소나: 역할과 정체성 부여
 - AI의 사고 방식과 응답 스타일을 결정
- 작업 : 구체적인 행동을 지시하기
 - 프롬프트의 핵심으로, 모호한 요청보다는 구체적이고 실행 가능한 지시를 하는 것이 중요
- 맥락: 배경 설명하기
 - 맥락 정보가 있으면 AI가 상황을 파악하고 적절한 답변을 생성함
- 형식: 출력 형태 지정하기
 - 결과물을 바로 활용할 수 있게 만드는 실용적인 요소

✓ 명확하고 구체적으로 지시하기

○ 예1

파이썬에 대해 설명해줘

○ 예2

너는 전문 프로그래밍 강사야.

초등 학생이 이해할 수 있도록, 프로그램 언어 파이썬에서 '변수'의 정의와 목적을 실생활 예시를 들어 설명해줘.

→ 답변의 완성도가 달라짐

✓ 예시를 들어 설명하기

○ 예1

신제품 출시 안내 이메일 제목을 몇 개 추천해줘.

○ 예2

신제품 출시 안내 이메일 제목을 몇 개 추천해줘

다음은 고객에게 발송하는 이메일 제목의 적절한 예시야.

이와 유사한 톤앤매너로 신제품 'Gemini Pro' 출시를 알리는 이메일 제목 5가지를 추천해줘.

예시 1: [신규 기능] 이제 더 스마트하게 작업하세요.

예시 2: 놓치지 마세요! 생산성을 향상시킬 새로운 업데이트 소식

✓ 충분히 많은 정보 제공하기

○ 예

다음 기사 내용을 세 개의 핵심 문장으로 요약해줘

[기사 전문] "인공지능(AI) 기술이 빠르게 발전하면서 우리 삶의 많은 부분이 변화하고 있습니다. 특히 자연어 처리 기술의 발전은 인간과 컴퓨터의 소통 방식을 혁신적으로 바꾸고 있으며, 이는 교육, 의료, 고객 서비스 등 다양한 분야에 적용되어 효율성을 높이고 있습니다.

...생략...

✓ 대답 형식 지정하기

○ 예1

주요 프로그래밍 언어 3개를 제시해줘.

○ 예2

주요 프로그래밍 언어 3개를 JSON 형식으로 제시해줘.

✓ 해야 할 일과 하지 말아야 할 일 알려주기

○ 예1

Google의 AI 모델 '제미나이(Gemini)'에 대해 설명해줘.

○ 예2

Google의 AI 모델 '제미나이(Gemini)'에 대해 설명해줘.

기술적인 내용은 제외하고, 일반인이 이해하기 쉽게 200자 이내로 작성해줘. 역사나 개발 과정은 포함하지 마.

✓ 프롬프트 소소한 팁

- 자연스러운 대화
- 대화 이어 나가기
- 문서 활용

K-디지털 트레이닝

프로젝트 컨텍스트 작성하기

[KB] IT's Your Life

✓ 제미나이 CLI의 진정한 힘

- 프로젝트 전체의 맥락을 이해하는 것
- 프롬프트만으로는 부족 → Gemini.md 필요

✓ Gemini.md

- 제미나이 CLI가 사용하는 컨텍스트 파일의 이름
- 메모리 또는 프로젝트 컨텍스트라고 부름
- 프로젝트마다 필요한 지식과 규칙을 적어둠
 - 제미나이의 행동 지침
 - 마크다운 형식으로 작성

✓ Gemini.md 파일 내용

- 프로젝트의 목표와 개요
 - 이 프로젝트는 무엇을 위한 것인가?
- 주요 기술 스택
 - 어떤 프로그래밍 언어, 프레임워크, 라이브러리를 사용하는가?
- 코딩 스타일 가이드
 - 변수명은 카멜 케이스를 사용하는가, 스네이크 케이스를 사용하는가?
- 중요한 규칙 및 제약 사항
 - 반드시 지켜야 할 규칙이나 피해야 할 패턴은 무엇인가?
- AI에게 부여할 페르소나
 - AI가 가질 역할이나 성격은 무엇인가?

Gemini.md

```
# Project: My Awesome TypeScript Library
```

```
## General Instructions:
```

- TypeScript 코드 작성 시 기존 코딩 스타일을 따를 것
- 모든 함수와 클래스에 JSDoc 주석 작성
- 가능하면 함수형 프로그래밍 패러다임 사용
- TypeScript 5.0과 Node.js 20+ 호환성 보장

```
## Coding Style:
```

- 들여쓰기는 2칸(space) 사용
- 인터페이스는 I로 시작 (예 : IUserService)
- 클래스의 private 멤버는 _로 시작
- 항상 strict equality(===, !==) 사용

```
## Specific Component: src/api/client.ts
```

- 모든 API 요청을 담당
- 신규 API 함수 추가 시 robust한 에러 처리 및 로깅 필수
- GET 요청은 반드시 fetchWithRetry 유틸리티 사용

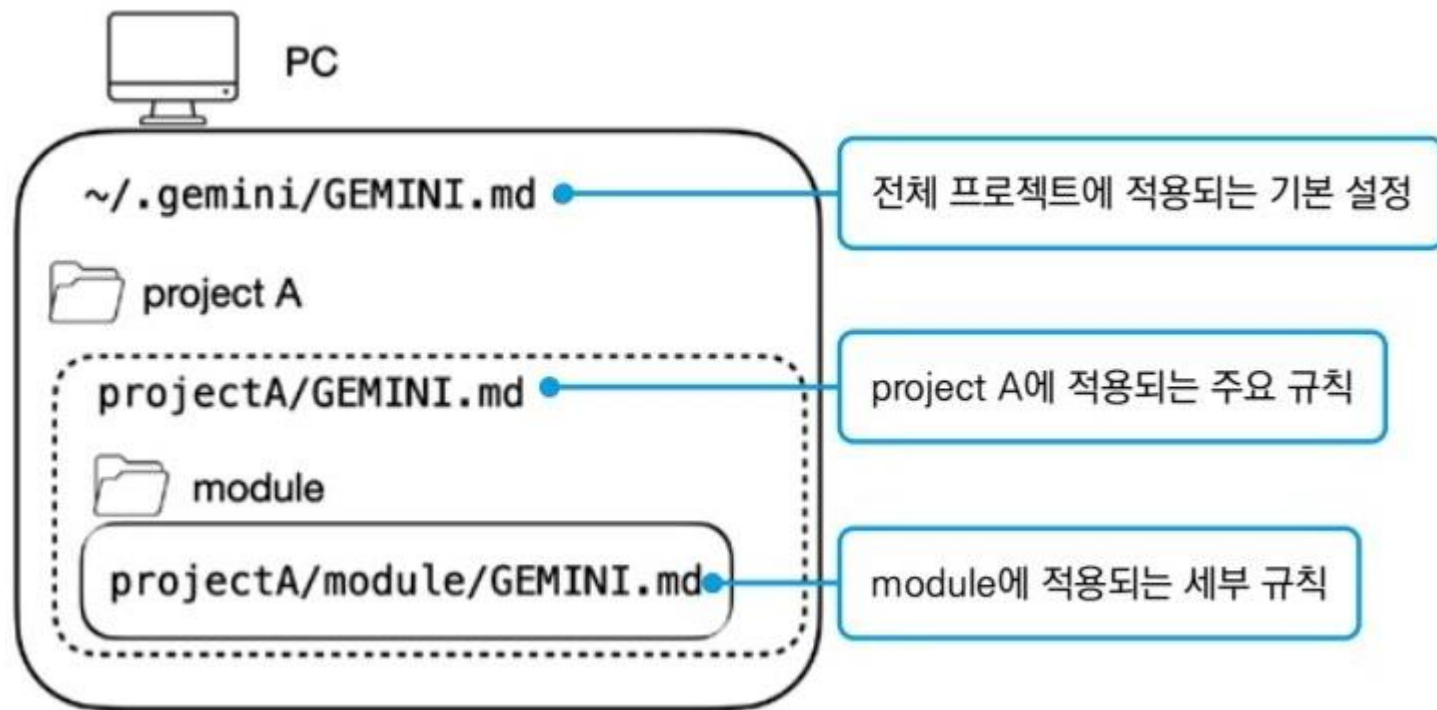
Gemini.md

Dependencies:

- 반드시 필요한 경우가 아니면 외부 라이브러리 도입 금지
- 새 의존성 추가 시 사유 명시

✓ 컨텍스트 파일의 적용 순서

- 여러 위치에서 순차적으로 탐색
- 더 구체적인 내용이 있을 경우 상위 파일의 내용을 덮어쓰거나 보완
- 컨텍스트 파일 탐색 및 적용 우선 순위
 1. 모듈 컨텍스트
 2. 프로젝트 컨텍스트
 3. 글로벌 컨텍스트



✓ /init 명령

- 현재 디렉터리를 분석하고 컨텍스트 파일을 자동 생성
- 기존 프로젝트에 제미나이 CLI를 접목하고 싶을 때 유용

✓ 구글, OpenAI, 엔트로픽의 컨텍스트 작성 가이드 참고

- 제미나이 CLI: bit.ly/46mLKZB
- 코덱스: bit.ly/4lYUbzZ
- 클로드 코드: bit.ly/4lOXHN7

→ 빠르게 새로운 프로젝트를 구성하고 싶은 경우 이 가이드를 토대로 제미나이 CLI에게 컨텍스트 생성 요청

이 가이드를 바탕으로 지금 프로젝트에 대한 컨텍스트를 작성해줘.

<< 가이드 복사 & 붙여 넣기 >>

→ 컨텍스트 작성 시 영어로 작성할지 한글로 작성할지 지정

(참고: 영어로 하면 토큰 소모를 줄일 수 있음)

K-디지털 트레이닝

내장 명령어 알아보기

[KB] IT's Your Life

✓ /about

- 현재 사용 중인 제미나이 CLI의 버전과 관련 정보를 확인

About Gemini CLI

CLI Version	0.39.1
Git Commit	4d73f3413
Model	gemini-3-flash-preview
Sandbox	no sandbox
OS	win32
Auth Method	Signed in with Google (gusukim@gmail.com)
Tier	Gemini Code Assist for individuals

✓ /chat

- 제미나이 CLI에 사용했던 프롬프트와 답변, 즉 대화 기록을 관리함
- 대화 저장하기
 - /chat save <태그명>
- 대화목록 출력
 - /chat list
- 불러오기
 - /chat resume <태그명>
- 예]
제미나이에 대해 설명해줘
<< 답변 >>
/chat save GEMINI
/chat list
/chat resume GEMINI

✓ /memory

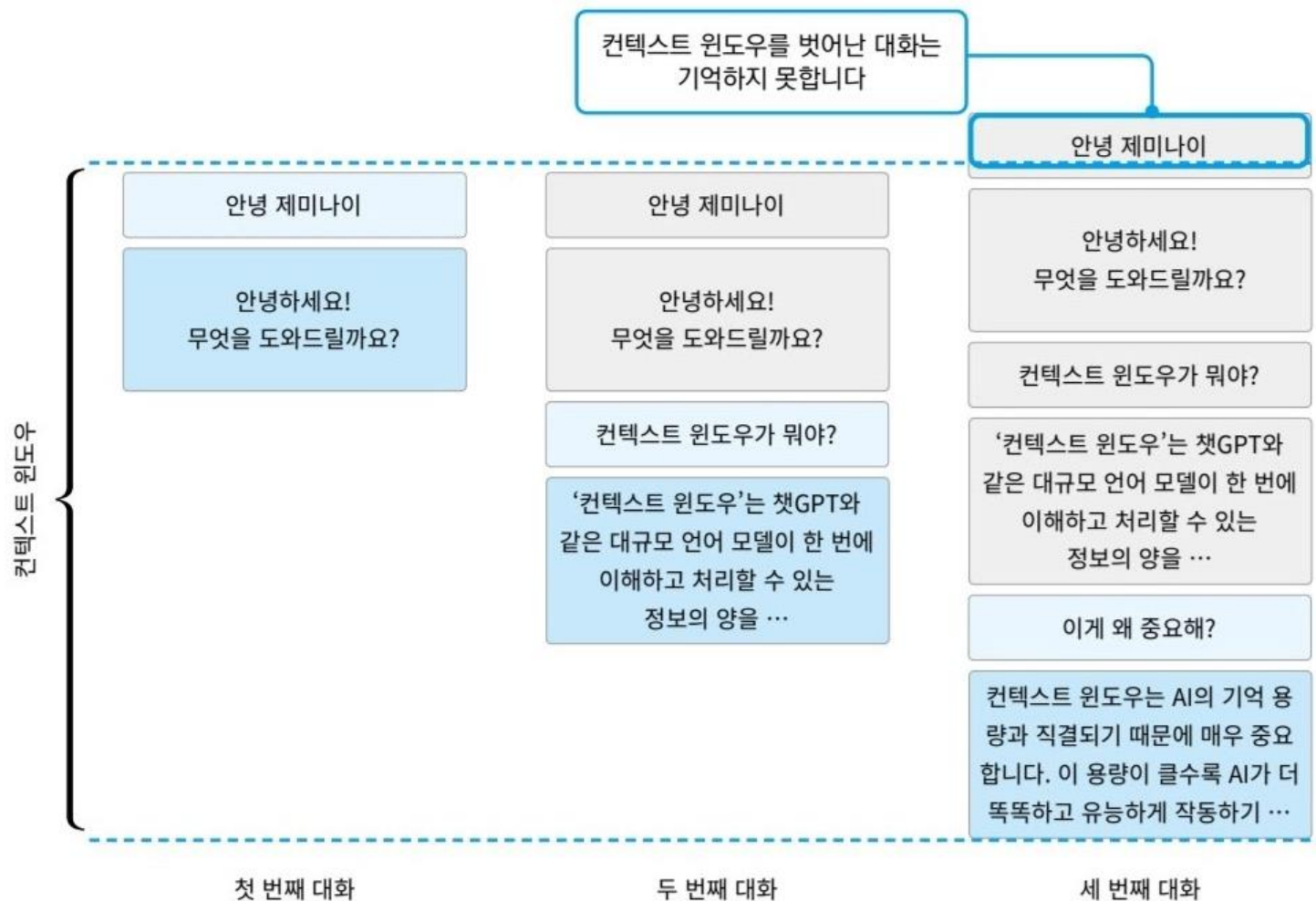
- 컨텍스트 확인 및 컨텍스트 니용 관리
- /memory show
 - 모든 컨텍스트를 한 번에 확인
- /memory refresh
 - 컨텍스트 파일을 다시 읽고 최신 정보로 갱신
- /memory add <텍스트>
 - 글로벌 컨텍스트 파일에 <텍스트>를 추가
 - 답변 스타일이나 언어처럼 모든 답변에 적용하고 싶은 설정을 추가

/memory add "한국어로 답변해줘"

/memory refresh

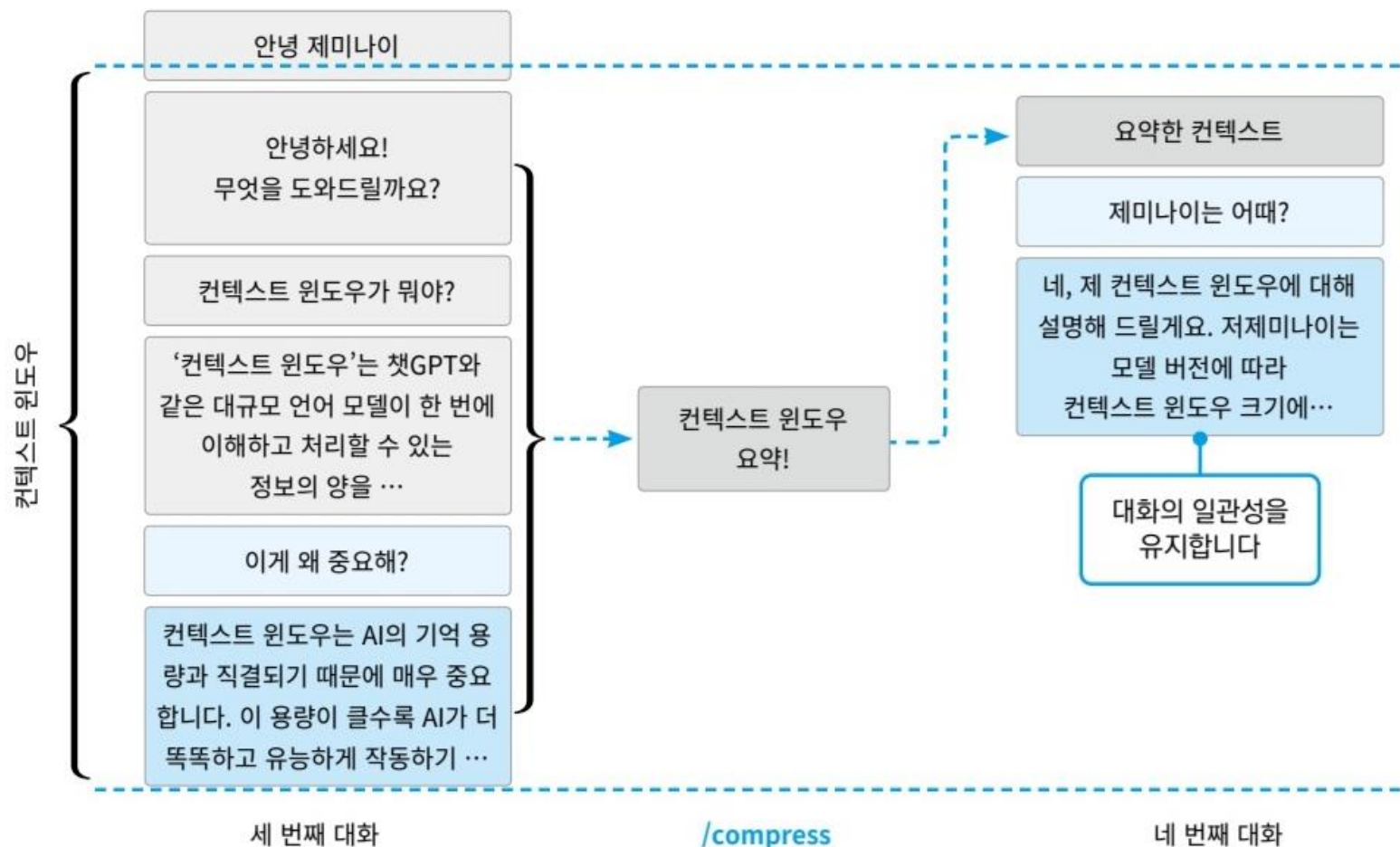
✓ /compress

- 현재 진행 중인 대화를 요약하고 대체
- 대화가 길어질 때 이방법을 사용해 사용하는 토큰 수를 조절
- 컨텍스트 윈도우
 - 최대 크기가 정해져 있음
 - 토큰 수로 표현
 - 오래된 대화 내용은 사라짐



✓ /compress

- 이전에 이어지던 대화의 주요 내용은 유지하면서 동시에 요약하여 사용되는 토큰 수를 축소
- 요약 과정에서 중요한 요소가 제외될 수 있으므로 신중하게 사용
→ 필요하다면 컨텍스트를 새롭게 추가함



✓ /theme

○ 색상 테마 변경

> Select Theme



1. ANSI Light
2. Ayu Light
- 3. Default Light
4. GitHub Light
5. Google Code Light
6. Solarized Light
7. Xcode Light
8. ANSI Dark (Incompatible)
9. Atom One Dark (Incompatible)
10. Ayu Dark (Incompatible)
11. Default Dark (Incompatible)
12. Dracula Dark (Incompatible)



(Use Enter to select, Tab to configure scope, Esc to close)

Preview

```
1 # function
2 def fibonacci(n):
3     a, b = 0, 1
4     for _ in range(n):
5         a, b = b, a + b
6     return a

1 - print("Hello, " + name)
1 + print(f"Hello, {name}!")
```


✓ /copy

- 가장 마지막으로 반환한 결과물을 클립보드에 복사

> explain about Gemini cli

+ I am Gemini, an AI-powered agent that functions as a command-line interface (CLI). My purpose is to bring the capabilities of a large language model directly to your terminal, helping you with a variety of tasks without leaving your command line.

You can interact with me to:

- * **Analyze code and projects:** I can read files, understand the structure of your project, and help you make sense of it.
- * **Modify code:** I can help you write, refactor, or debug code by directly editing files.
- * **Run commands:** I can execute shell commands to build, test, or run your projects.
- * **Answer questions:** I can answer general knowledge questions or questions specific to your project's context.
- * **Automate tasks:** I can help automate repetitive tasks by chaining commands and operations.

I use a GEMINI.md file within your project to learn its specific context, which allows me to provide more tailored and effective assistance.

> /copy

/copy 명령어를 입력합니다

{last output copied to the clipboard}

클립보드에 복사되었습니다

✓ /init

- 현재 작업 중인 디렉터리에 GEMINI.md 파일을 생성
- 현재 작업 중인 디렉터리를 자동으로 분석하여 맞춤형 컨텍스트 파일을 생성
- 프로젝트에서 제미나이 CLI를 처음 실행했을 때 사용

```
? WriteFile Writing to GEMINI.md ←

... first 30 lines hidden ...
30 **Example:**
31
32 *   This project follows the [Contributor Covenant](
    https://www.contributor-covenant.org/) code of conduct.
33 *   Please run `npm run lint` before committing.
34 *   All new features should be accompanied by unit tests.

Apply this change?

● 1. Yes, allow once
   2. Yes, allow always
   3. No (esc)
```

GEMINI.md를
자동으로 작성합니다

✓ 명령어

명령어	설명
/about	버그 리포트 작성 또는 현재 사용 중인 모델과 인증 방식 등을 확인할 때 사용
/auth	인증 방식 전환
/bug	CLI 사용 중 오류 발생 시 깃허브 이슈로 버그 제보
/chat	대화를 저장하거나 재개
/compress	컨텍스트 요약
/copy	마지막 답변 전체를 클립보드에 복사
/directory(/dir)	현재 작업 공간 외에 다른 디렉터리에 있는 파일을 참조 시 컨텍스트에 추가
/docs	제미나이 CLI 공식 문서 열람
/editor	코드 수정이나 파일 비교 시 사용할 외부 텍스트 에디터 지정
/extensions	현재 세션에서 사용 가능한 확장 기능 목록 확인
/help(/?)	사용 가능한 모든 명령어와 단축키 목록 표시
/ide	통합 개발 환경 연동 설정
/init	현재 디렉터리를 분석하여 최적의 GEMINI.md 파일을 자동으로 생성

✓ 명령어

명령어	설명
/mcp	Model Context Protocol 서버 및 사용 가능한 도구 나열
/memory	로드된 메모리 컨텍스트 관리
/model	현재 사용 중인 모델을 표시하거나 다른 모델로 변경
/privacy	데이터 수집 및 활용 정책을 확인하고 데이터 수집 허용 여부를 설정
/quit(/exit)	Gemini CLI 세션을 종료
/settings	대화형 인터페이스로 .gemini/settings.json 내용 수정
/setup-github	깃허브 리포지토리에서 PR 리뷰, 이슈 관리 등을 자동화할 수 있도록 연동 작업 수행
/stats	현재 세션의 토큰 사용량 및 통계 표시
/terminal-setup	IDE 내 터미널에서 줄 바꿈을 위한 키 조합 설정 (기본값: Ctrl + J)
/theme	CLI 인터페이스의 색상 테마 변경
/tools	모델이 사용할 수 있는 내장 도구 목록 표시
/vim	터미널에서 Vim 스타일의 키보드 입력 방식을 사용하도록 전환
/clear	터미널 화면 지우기

✓ !셸 명령어

- 셸 명령을 바로 실행

!ls

!dir

✓ @<파일경로>, @<url>

- 파일 내용을 가져와 대화에 활용
- 파일경로 입력시 자동완성 기능 제공
- 경로상에 공백이 있는 경우 \로 공백을 이스케이프
- 예

@src/utils.js 이 코드의 문제점을 찾아주고, 더 효율적으로 개선해줘.

@meeting_notes.md 오늘 회의록의 핵심 내용을 세 줄로 요약해줘.

@package.json 이 프로젝트에 설치된 의존성 라이브러리를 알려주고, 각 라이브러리의 용도를 설명해줘

@http://github.com/google-gemini/gemini-cli/blob/main/docs/cli/commands.md 내용을 정리해줘

web_fetch를 사용해 @http://github.com/google-gemini/gemini-cli/blob/main/docs/cli/commands.md 내용을 정리해줘

✓ 참고

- 바이너리로 클립보드에 복사한 경우(이미지 복사) Alt+V로 대화에 추가 가능
 - 해당 내용을 임시파일로 저장하고, 그 파일 내용을 사용함.

K-디지털 트레이닝

내장 도구 알아보기

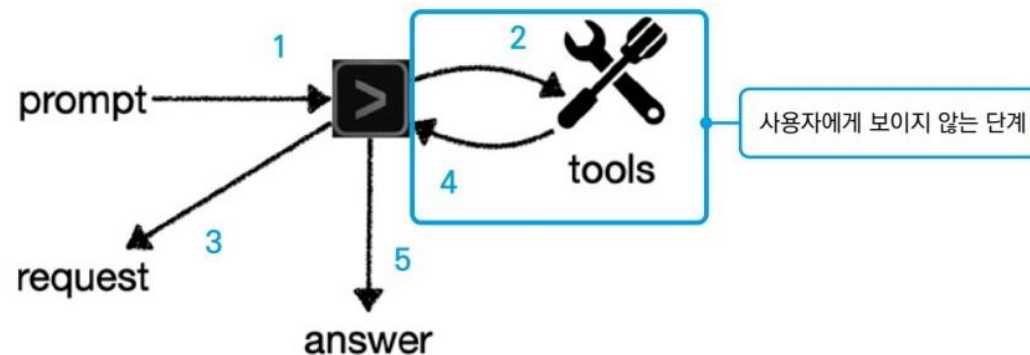
[KB] IT's Your Life

✓ 내장 도구

- 사용자가 직접 사용하지 않음
- 제미나이 CLI가 문맥을 유추해 자동으로 실행하거나
- 프롬프트에 어떤 도구를 사용하라고 지시 가능

○ 제미나이 CLI의 내장도구 사용 단계

1. 사용자가 프롬프트를 입력
2. 프롬프트와 사용 가능한 도구 정보가 제미나이 CLI에 전달
3. 제미나이가 프롬프트를 해석해 도구 실행이 필요하면 도구의 정보를 알리고 사용 여부를 사용자에게 확인 받음
4. 제미나이가 도구를 사용하여 작업을 수행
5. 작업 및 답변 생성



✓ 정확한 정보가 필요할 땐 검색 도구

○ web_fetch

- 웹 페이지의 정보를 추출하고 분석하는 도구
- 최대 20개의 URL을 한 번에 처리 가능
- 프롬프트에 URL과 함께 요약, 비교 등의 키워드를 입력하면 자동으로 사용

▪ 예

@<http://github.com/google-gemini/gemini-cli/blob/main/docs/cli/commands.md> 내용을 정리해줘

Action Required

? WebFetch Processing URLs and instructions from prompt: "Summarize the contents of <https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/commands.md>..."

Summarize the contents of <https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/commands.md>, focusing on the available CLI commands and their primary functions.

URLs to fetch:

- <https://raw.githubusercontent.com/google-gemini/gemini-cli/main/docs/cli/commands.md>,

Do you want to proceed?

1. Allow once
- 2. Allow for this session
3. No, suggest changes (esc)

GoogleSearch Searching the web for: "Gemini CLI commands documentation github google-gemini"
 → Search results for "Gemini CLI commands documentation github google-gemini" returned.

+ 제공해주신 링크가 현재 유효하지 않아(404), 최신 정보를 바탕으로 Gemini CLI의 주요 명령어 체계를 정리해 드립니다. Gemini CLI는 크게 슬래시(/), 앳(@), 느낌표(!) 세 가지 접두사를 사용합니다.

1. 슬래시 명령어 (/ - 시스템 제어)

CLI 세션 자체를 관리하거나 설정하는 메타 명령어입니다.

- * **/help**: 도움말 메뉴 및 사용 가능한 명령어 목록을 표시합니다.
- * **/chat**: 세션 상태 관리 (예: **/chat save <tag>**, **/chat resume <tag>**, **/chat list**).
- * **/clear**: 터미널 화면과 현재 세션의 대화 기록을 삭제합니다.
- * **/settings**: 대화형 에디터를 통해 CLI 설정을 수정합니다.
- * **/tools**: 사용 가능한 도구(파일 읽기, 웹 검색 등) 목록을 확인합니다.
- * **/memory**: 세션 간에 유지되는 기억(facts)을 관리합니다.
- * **/bug**: 터미널에서 즉시 GitHub 이슈를 생성합니다.
- * **/plan**: 복잡한 변경 전 설계를 위한 플랜 모드를 실행합니다 (읽기 전용).

2. 컨텍스트 주입 (@ - 파일/디렉토리 참조)

Gemini에게 특정 파일이나 폴더의 내용을 컨텍스트로 전달할 때 사용합니다.

- * **@파일명**: 특정 파일을 대화에 포함 (예: **@README.md** "이 파일 요약해줘")
- * **@폴더명/**: 디렉토리 전체 구조나 내용을 참조 (예: **@src/** "이 구조 설명해줘")
- * **여러 파일 지정**: **@package.json @vitest.config.ts** "의존성 체크해줘"

3. 셸 실행 (! - 시스템 명령)

CLI 세션을 나가지 않고 터미널 명령어를 직접 실행합니다.

- * **! npm test**: 테스트 실행
- * **! git status**: 현재 Git 상태 확인
- * **! ls -la**: 현재 디렉토리 파일 목록 확인

추가 기능

- * **커스텀 명령어**: **~/.gemini/commands/** 경로에 **.toml** 파일을 추가하여 나만의 단축 명령어를 만들 수 있습니다. (예: **refactor.toml** 생성 시 **/refactor** 명령 사용 가능)
- * **GEMINI.md**: 프로젝트 루트에 이 파일을 두어 해당 프로젝트만의 특별한 규칙이나 기술적 맥락을 AI에게 항상 주입할 수 있습니다.

✓ 정확한 정보가 필요할 땐 검색 도구

○ google_web_search

- 실시간으로 구글 검색 수행
- 출처 기반으로 생성된 결과이므로 정확도가 높고 환각 현상을 보완
- 최종적으로 검색 결과를 요약한 텍스트를 반환

✓ 파일 관리를 위한 파일 시스템 도구

○ `list_directory`

- 특정 디렉터리에 있는 파일과 디렉터리 목록을 가져옴

- 예

`list_directory` 도구를 사용해 현재 디렉터리에 어떤 파일들이 있는지 확인해 줘, `.png`와 `.md` 파일은 무시해

✓ 파일 관리를 위한 파일 시스템 도구

○ read_file

- 특정 파일의 내용을 가져오는 도구
- 텍스트, 이미지, PDF 파일 지원

▪ 예

prompt.pdf에는 어떤 내용이 있지?

✓ 파일 관리를 위한 파일 시스템 도구

○ write_file

- 파일을 새로 만들거나 기존 파일 내용 덮어 쓰기
- 사용자 허가 요청

- 예

2개의 숫자를 입력받아 합을 구하는 파이썬 코드를 sum.py에 작성해줘

✓ 다양한 내장 도구

구분	도구명 (내부 이름)	기능
파일 시스템	list_directory	지정한 디렉터리의 파일과 하위 디렉터리 목록 반환
	read_file	지정된 파일의 내용을 읽어 반환
	write_file	파일 내용을 작성하거나 덮어쓰기
	write_todos	복잡한 작업을 위한 단계별 하위 작업 생성
	glob	glob 패턴과 일치하는 파일 검색
	search_file_content	정규표현식으로 파일 내부 텍스트 검색
	replace	파일 내 특정 텍스트 블록을 다른 내용으로 교체
셸 명령어	run_shell_command	셸 명령어를 직접 실행
웹 검색	google_web_search	구글 검색 결과 요약
	web_fetch	지정한 URL의 웹페이지 콘텐츠 분석
메모리 관리	save_memory	중요한 정보를 GEMINI.md에 저장

✓ 제미나이 CLI 커맨드 라인

○ -m 옵션

- 제미나이 모델 지정

`gemini -m "gemini-2.5-flash"`

○ -p 옵션

- 프롬프트를 내장 도구의 인자로 직접 전달
- 채팅이 아닌 일회성 명령

`gemini -p "google_web_search('최신 AI 뉴스를 소개해줘')"`

○ --show_memory_usage

- 제미나이 CLI 상태 표시줄 오른쪽 아래에 현재 메모리 사용량을 표시

`gemini --show_memory_usage`

✓ 세미나이 CLI 커맨드 라인

○ -v, --version

- 세미나이 CLI 버전 출력

gemini -v

gemini --version

○ -y, --yolo

- 모든 도구 호출 및 작업을 자동으로 승인

gemini -y

gemini --yolo

○ --help

옵션	설명
-d, --debug	디버그 모드로 실행
-m, --model	사용할 모델 지정
-p, --prompt	표준 입력에 추가할 프롬프트 지정
-i, --prompt-interactive	주어진 프롬프트를 실행하고, 인터랙티브 모드로 실행
-s, --sandbox	샌드박스 실행 여부 설정
-y, --yolo	모든 작업을 자동으로 수락 (yolo 모드)
-e, --extensions	사용할 익스텐션 설정 (기본값은 모든 익스텐션 사용)
-l, --list-extensions	설치된 익스텐션 목록 출력
-r, --resume	이전 채팅 세션에 이어서 작업
-o, --output-format	CLI 출력 형식 설정 (text, json, stream-json)
-v, --version	프로그램 버전 표시
-h, --help	도움말 메시지 표시

K-디지털 트레이닝

설정 파일 알아보기

[KB] IT's Your Life

✓ 제미나이 CLI 설정 우선순위

1. 커맨드라인 인자
2. 환경 변수
3. Project 설정 파일: <project>/.gemini/settings.json
4. User 설정 파일: ~/.gemini/settings.json
5. 기본값: 별도로 설정하지 않은 경우

~/.gemini/settings.json

```
{
  "hasSeenIdeIntegrationNudge": true,
  "security": {
    "auth": {
      "selectedType": "oauth-personal"
    }
  },
  "ui": {
    "theme": "Default Light"
  },
  "ide": {
    "hasSeenNudge": true
  }
}
```

- ✓ 현재 프로젝트에 settings.json 만들기
 - <프로젝트>/.gemini/settings.json 파일 생성

 <프로젝트>/.gemini/settings.json

```
{
  "ui": {
    "theme": "Ayu",
    "hideTips": true
  }
}
```

☑ settings.json 주요 설정 항목

설정		설명
general	previewFeatures	미리보기 기능 활성화
	preferredEditor	파일 열기에 선호하는 에디터
	vimMode	Vim 키 바인딩 활성화
	checkpointing.enabled	세션 복구용 체크포인트 활성화
	enablePromptCompletion	AI 기반 프롬프트 자동 완성 제안
ui	theme	UI 색상 테마
	hideTips	유용한 팁 숨기기
	hideBanner	애플리케이션 배너 숨기기
	showMemoryUsage	메모리 사용 정보 표시
	showCitations	생성된 텍스트에 인용 표시
ide	enabled	IDE 통합 모드 활성화

☑ settings.json 주요 설정 항목

설정		설명
model	name	사용할 제미나이 모델명
context	fileName	메모리에 로드할 컨텍스트 파일명
	includeDirectories	작업 공간에 포함할 추가 디렉토리
	fileFiltering.respectGitIgnore	.gitignore 파일 준수 여부
tools	autoAccept	안전한 도구 호출(읽기 전용 등) 자동 수락
	sandbox	샌드박스 활성화 여부 또는 프로필 경로
	shell.showColor	셸 출력에 색상을 표시합니다.
security	auth.selectedType	현재 인증 유형

- 일부 옵션은 /settings 내부 명령으로 설정

✓ .env에 환경 변수 설정하기

- 프로그램의 동작 방식이나 API 키, 비밀번호 등 민감한 정보를 안전하게 전달하는 데 사용

환경 변수	설명
GEMINI_API_KEY	API 연동에 사용하는 제미나이 API 키
GEMINI_MODEL	기본으로 사용할 제미나이 모델 지정
GOOGLE_API_KEY	Google Cloud API연동에 사용하는 키
GOOGLE_CLOUD_PROJECT	사용할 Google Cloud 프로젝트 ID 지정
GOOGLE_APPLICATION_CREDENTIALS	Google 서비스 계정 인증 파일 경로 지정
GEMINI_SANDBOX	샌드박스 환경 사용 여부 및 실행 방식을 지정 (“docker”, “podman”, true 등)
DEBUG, DEBUG_MODE	디버그 모드를 활성화
NO_COLOR	CLI 출력에서 색상을 사용하지 않도록 설정
CLI_TITLE	CLI의 제목을 원하는 이름으로 변경

변수명=값

K-디지털 트레이닝

안전하게 작업하기

[KB] IT's Your Life

✓ 체크포인트 기능 활성화

○ 파일 시스템을 수정하는 도구(write_file, replace 등)를 실행할 때 다음 정보를 담은 체크포인트를 자동으로 생성

- 깃 스냅샷
 - 프로젝트의 전체 파일 상태를 특별한 깃 저장소(~/.gemini/history/<project_hash>)에 커밋함
 - 사용자의 실제 프로젝트 깃 저장소와 완전히 별개로 동작
- 기존 작업에 영향을 주지 않음

```
[cha@chaui-MacBookAir gemini_proj % ls -al ~/.gemini/history/
total 0
drwxr-xr-x  4 cha  staff  128  10  1 14:47 .
drwxr-xr-x 11 cha  staff  352  10 17 09:42 ..
drwxr-xr-x  5 cha  staff  160   8 22 17:28 5dfe6dd92b1de804e4764152f378384958c6d8ffd448b361679e81c4b9580d38
drwxr-xr-x  5 cha  staff  160  10  1 14:47 f8396ae488743a1b2c28b0e9979ec02ec0bc584c5f4a259ebbd1c55c5751e90f
[cha@chaui-MacBookAir gemini_proj % ls -al ~/.gemini/history/5dfe6dd92b1de804e4764152f378384958c6d8ffd448b361679e81c4b9580d38
total 16
drwxr-xr-x  5 cha  staff  160   8 22 17:28 .
drwxr-xr-x  4 cha  staff  128  10  1 14:47 ..
drwxr-xr-x 12 cha  staff  384   8 22 17:32 .git
-rw-r--r--  1 cha  staff   86   8 22 17:28 .gitconfig
-rw-r--r--  1 cha  staff  109   8 22 17:28 .gitignore
cha@chaui-MacBookAir gemini_proj %
```

프로젝트 고유 번호입니다

내부는 깃 저장소 형태입니다

- 대화기록
- 툴 호출 정보

☑ settings.json 파일을 통한 체크포인트 활성화

📄 ~/.gemini/settings.json

```
{  
  "general" : {  
    "checkpointing" : {  
      "enabled" : true  
    }  
  }  
}
```

- 체크포인트 기능을 켜면 /restore 명령어가 활성화됨

✓ /restore 체크 포인트 복원

- 체크포인트 복원 및 관리 명령
- 체크포인트는 대화마다 자동 생성
- 작업시간은 협정 세계시 UTC를 사용
- 모든 체크포인트는 제미나이 CLI 세션 종료 시 사라짐
- /resotre
 - 저장된 체크포인트 목록 확인
- /restore <checkpoint_file>
 - 지정한 체크포인트로 복원
 - checkpoint_file명은 자동완성 기능 지원

✓ 보안 강화를 위한 격리 환경, 샌드박싱

- 셸 명령 실행이나 파일 수정 등 시스템에 위험할 수 있는 작업을 실제 시스템을 보호하면서 다양한 실험과 자동화 작업을 안심하고 수행 할 수 있음
- 기본적으로 꺼져 있음
- 활성화 명령
 - `gemini --sandbox`
 - `gemini -s`
- 활성화 설정
 - `GEMINI_SANDBOX=true`
 - ```
{
 "tools" : {
 "sandbox": true
 }
}
```

K-디지털 트레이닝

# 커스텀 도구 만들기

---

[KB] IT's Your Life

## ✓ 커스텀 도구

- 제미나이 CLI에서 기본적으로 제공하지 않는 도구를 추가하는 기능
- 길고 복잡한 프롬프트를 사용하지 않더라도 정해진 흐름에 따라 파일 생성, 메시지 전송, 데이터 변환 등 반복 작업 수행
- /tools
  - 현재 제미나이 CLI에 사용 가능한 도구 확인

Available Gemini CLI tools:

- Activate Skill (activate\_skill)
- Ask User (ask\_user)
- Edit (replace)
- Enter Plan Mode (enter\_plan\_mode)
- FindFiles (glob)
- GoogleSearch (google\_web\_search)
- Invoke Subagent (invoke\_agent)
- List Background Processes (list\_background\_processes)
- Read Background Output (read\_background\_output)
- ReadFile (read\_file)
- ReadFolder (list\_directory)
- SaveMemory (save\_memory)
- SearchText (grep\_search)
- Shell (run\_shell\_command)
- WebFetch (web\_fetch)
- WriteFile (write\_file)

## ✓ 커스텀 도구

### ○ 셸 스크립트로 사용자가 만들어서 추가한 도구

- 윈도우 환경에서 실행하기 어려움
- 제미나이 CLI를 Git Bash에서 실행하면 가능

### ○ 커스텀 도구 관련 파일

- Discovery Script
  - 프로젝트에서 사용할 커스텀 도구를 제미나이 CLI에 도구의 이름, 설명, 파라미터 등을 알려주는 스크립트
- Execution Script
  - 제미나이 CLI가 특정 커스텀 도구를 실행하라고 요청했을 때 실제 작업을 처리하는 스크립트
- settings.json
  - 위 두 스크립트를 어디서 찾아 실행해야 하는지 제미나이 CLI에 알려주는 설정 파일
- GEMINI.md
  - 커스텀 도구 사용 지침 제공



## ✓ 간단한 커스텀 도구 생성하기

### ○ echo 도구

#### bin/get\_tools (Discovery Script)

```
#!/bin/bash
cat <<EOF
[
 {
 "name": "echo",
 "description": "A tool that returns the input text as it is.",
 "parameters": {
 "type": "object",
 "properties": {
 "text": {
 "type": "string",
 "description": "The string to return."
 }
 },
 "required": ["text"]
 }
 }
]
EOF
```

## bin/call\_tool (Execution Script)

```
#!/bin/bash
FUNC_NAME="$1"
shift
case "$FUNC_NAME" in
 echo)
 # Read parameters from stdin
 read INPUT_JSON
 # Extract text using jq
 TEXT=$(echo "$INPUT_JSON" | jq -r '.text')
 # Output result as JSON
 echo "{\"result\": \"$TEXT\"}"
 ;;
 *)
 echo "{\"error\": \"Unknown function: $FUNC_NAME\"}"
esac
```

## 두 스크립트에 실행 권한 추가하기

**chmod +x get\_tools**

**chmod +x call\_toll**

윈도우 Git Bash)  
<https://github.com/jqlang/jq/releases/latest/download/jq-win64.exe>을  
다운로드 받아 C:\Program Files\Git\usr\bin에 jq.exe로 복사  
→ 관리자 권한 필요

## .gemini/settings.json

```
{
 "tools" : {
 "toolDiscoveryCommand": "bin/get_tools",
 "toolCallCommand": "bin/call_tool"
 }
}
```

## GEMINI.md

## Echo

- echo 도구는 다음과 같이 사용합니다.
- 예시: echo "Hello, world!"
- 입력한 따옴표 안의 문자열을 그대로 반환합니다.

## ✓ 사용하기

### ○ 프롬프트

- `echo(text="Hello, World!")`

```
> echo(text="Hello World")
```

```
✓ Shell echo '{"text": "Hello World"}' | bash bin/call_tool echo
{"result": "Hello World"}
█
```

## ✓ 참고

- 만들고 싶은 도구를 제미나이 CLI에게 알려주고 코드 작성을 지시하여 스크립트 생성 가능
- 제미나이 CLI는 커스텀 도구 보다는 MCP, 확장 기능 사용을 먼저 고려함
- 사용자가 커스텀 도구를 사용할 것이라 가정하지 않음
- 사용하는 AI 모델에 따라서 원하는 도구를 사용하지 못할 수 있음